

Searching

DSA27 — Lecture 08 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Week 8 · Fall 2026

Where We Are

From structures to algorithms

Weeks 4–7 built **structures**: dynamic array, linked list, stack, queue.

Weeks 8–10 are about **algorithms** on the simplest of them, the array:

- **Week 8**: find a value — **searching**
- **Weeks 9–10**: put the values in order — **sorting**

Today's question: **what does it buy you to keep an array sorted?**

1. Linear search: the baseline
2. Binary search: halving — and why it is so easy to get wrong
3. `lower_bound` and `upper_bound`: positions, duplicates, insertion
4. Jump, exponential and interpolation search
5. Measured: eight functions, one picture
6. When is sorting first worth it?

Linear Search

Look at everything

Linear search: no order needed, and no shortcut — $O(n)$

look at each element in turn



`linear_search(values, 8) → 4`
5 comparisons here; n when the target is last or missing

- Works on **any** sequence, sorted or not.
- Found at index i : $i + 1$ comparisons. Missing: n . Worst case $O(n)$.

Binary Search

Halve what is left

Binary search for 41: each comparison discards half of what is left

step 1 16 left	<table><tr><td>3</td><td>6</td><td>8</td><td>12</td><td>15</td><td>19</td><td>21</td><td>24</td><td>27</td><td>31</td><td>34</td><td>38</td><td>41</td><td>45</td><td>48</td><td>52</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr></table>	3	6	8	12	15	19	21	24	27	31	34	38	41	45	48	52	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	lo=0 hi=15 mid=7	24 < 41 → lo = 8
3	6	8	12	15	19	21	24	27	31	34	38	41	45	48	52																				
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																				
step 2 8 left	<table><tr><td>3</td><td>6</td><td>8</td><td>12</td><td>15</td><td>19</td><td>21</td><td>24</td><td>27</td><td>31</td><td>34</td><td>38</td><td>41</td><td>45</td><td>48</td><td>52</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr></table>	3	6	8	12	15	19	21	24	27	31	34	38	41	45	48	52	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	lo=8 hi=15 mid=11	38 < 41 → lo = 12
3	6	8	12	15	19	21	24	27	31	34	38	41	45	48	52																				
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																				
step 3 4 left	<table><tr><td>3</td><td>6</td><td>8</td><td>12</td><td>15</td><td>19</td><td>21</td><td>24</td><td>27</td><td>31</td><td>34</td><td>38</td><td>41</td><td>45</td><td>48</td><td>52</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr></table>	3	6	8	12	15	19	21	24	27	31	34	38	41	45	48	52	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	lo=12 hi=15 mid=13	45 > 41 → hi = 12
3	6	8	12	15	19	21	24	27	31	34	38	41	45	48	52																				
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																				
step 4 1 left	<table><tr><td>3</td><td>6</td><td>8</td><td>12</td><td>15</td><td>19</td><td>21</td><td>24</td><td>27</td><td>31</td><td>34</td><td>38</td><td>41</td><td>45</td><td>48</td><td>52</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr></table>	3	6	8	12	15	19	21	24	27	31	34	38	41	45	48	52	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	lo=12 hi=12 mid=12	41 = 41 → found at 12
3	6	8	12	15	19	21	24	27	31	34	38	41	45	48	52																				
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																				

Compare the target with the **middle** element. Equal: found. Smaller: it can only be on the left. Larger: only on the right.

The algorithm, and the promise it keeps

Keep two indices, `lo` and `hi`: the part still in play is `values[lo..hi]`, both ends included.

1. `lo = 0, hi = n - 1`
2. while `lo <= hi`: `mid = (lo + hi) // 2`
 - `values[mid] == target` → return `mid`
 - `values[mid] < target` → `lo = mid + 1`
 - otherwise → `hi = mid - 1`
3. return `-1`

Invariant: if the target is anywhere, it is in `values[lo..hi]`.

How many comparisons?

Each comparison halves the range: after k comparisons at most $n/2^k$ remain.

n	at most $\lfloor \log_2 n \rfloor + 1$ comparisons
1,000	10
1,000,000	20
1,000,000,000	30

Binary search is $O(\log n)$ — a thousand times more data costs 10 more comparisons.

Easy to state, hard to get right

- while $lo < hi$ with an inclusive $hi \rightarrow$ **misses** the last candidate.
- $hi = mid$ with an inclusive $hi \rightarrow$ can **loop for ever**.
- $lo = mid \rightarrow$ can **loop for ever** when $hi = lo + 1$.
- $mid = (lo + hi) / 2$ in Java or C $\rightarrow$ **overflows** when $lo + hi$ exceeds the largest int.

Recursive binary search

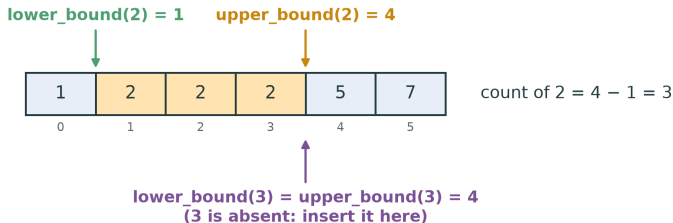
Same idea: search the half that can hold the target, **by a call**.

- $O(\log n)$ time, but also $O(\log n)$ **stack** — the loop is $O(1)$ space.
- Pass `lo` and `hi` down. **Never slice**: `values[mid + 1:]` copies — $O(n)$ per call.

Bounds: Searching for Positions

lower_bound and upper_bound

Bounds are positions between elements: where x would be inserted

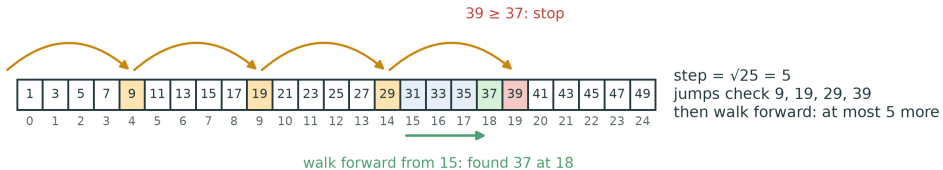


- $\text{lower_bound}(v, x)$: the first index with $v[i] \geq x$
- $\text{upper_bound}(v, x)$: the first index with $v[i] > x$
- Both may return $\text{len}(v)$. Neither returns -1 .

Three More Searches

Jump search: $O(\sqrt{n})$

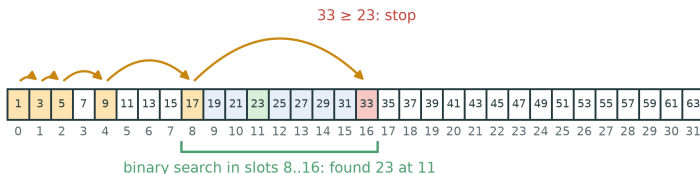
Jump search: jump $\sqrt{n}$ at a time, then walk forward inside one block



Jump ahead $\sqrt{n}$ at a time until you pass the target; then walk forward inside that one block.

Exponential search: $O(\log i)$

Exponential search: double the bound, then binary search inside it

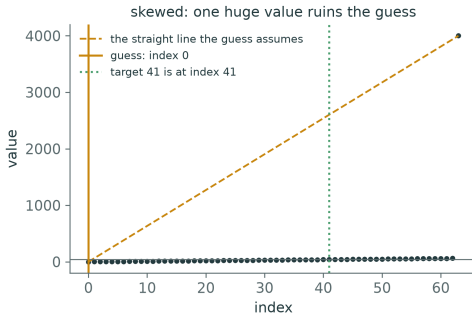
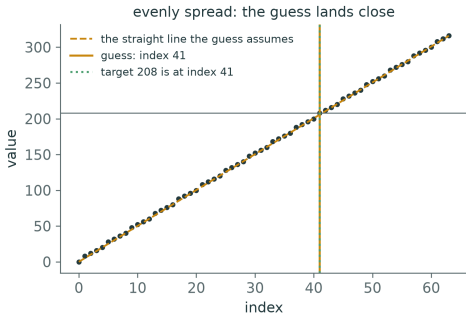


bounds 1, 2, 4, 8, 16:
 $O(\log i)$ steps, where i
is the answer's index

Check slots 1, 2, 4, 8, ... until one passes the target; then binary search between the last two bounds.

Interpolation search: guess, don't halve

Interpolation search guesses where the target should be — if the values are evenly spread



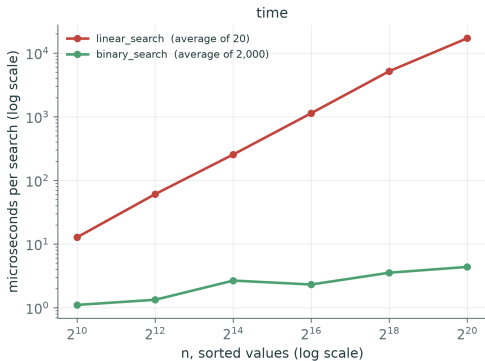
$$\text{pos} = \text{lo} + \frac{(\text{target} - v[\text{lo}]) (\text{hi} - \text{lo})}{v[\text{hi}] - v[\text{lo}]}$$

Evenly spread values: $O(\log \log n)$ on average. Skewed: $O(n)$.

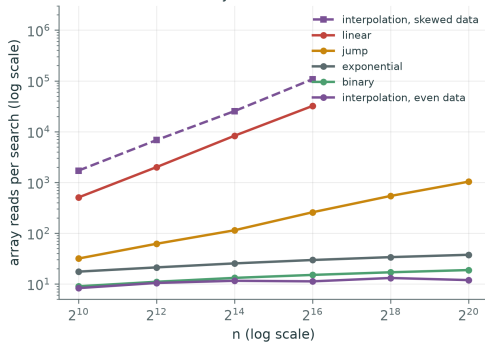
Measured

Eight functions, one picture

Measured: what a sorted invariant buys



work: how many elements each search reads



When is sorting first worth it?

Binary search needs sorted data, and sorting costs $O(n \log n)$ (Weeks 9–10).

k searches on n values	Cost
linear search, k times	$O(kn)$
sort once, then binary search k times	$O(n \log n + k \log n)$

Sorting wins once **k is larger than about** $\log n$ — for a million values, after a few dozen searches.

The eight functions

Function	Needs	Time	Note
linear_search	nothing	$O(n)$	first match
binary_search	sorted	$O(\log n)$	loop, $O(1)$ space
binary_search_recursive	sorted	$O(\log n)$	$O(\log n)$ stack; never slice
lower_bound	sorted	$O(\log n)$	first $v[i] \geq x$; may be len
upper_bound	sorted	$O(\log n)$	first $v[i] > x$; may be len
jump_search	sorted	$O(\sqrt{n})$	forward only
exponential_search	sorted	$O(\log i)$	unknown length; near the front
interpolation_search	sorted, even	$O(\log \log n)$ avg, $O(n)$ worst	guesses

This Week

Exercises: dsa/searching.py

Function	The trap
<code>linear_search</code>	unsorted input; return the first match
<code>binary_search</code>	<code>lo <= hi</code> ; <code>mid ± 1</code> ; <code>-1</code> when missing
<code>binary_search_recursive</code>	pass <code>lo, hi</code> ; no slices
<code>lower_bound, upper_bound</code>	half-open <code>[lo, hi)</code> ; <code>>=</code> against <code>></code>
<code>jump_search</code>	<code>math.isqrt</code> ; the last, partial block
<code>exponential_search</code>	slot 0 first; bound capped at <code>len - 1</code>
<code>interpolation_search</code>	equal ends; target outside <code>[v[lo], v[hi]]</code>

```
pytest tests/test_searching.py -v
```

Homework 8 — before Lecture 09

1. **Implement** `dsa/searching.py` until all forty tests pass.
2. **Trace** `binary_search` on `[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]` for 23 and for 60, as tables of `lo`, `hi`, `mid`.
3. **Measure** linear against binary search in `notebooks/08-searching.ipynb`, and count reads with a counting sequence, as in the figure.
4. **Break it.** Change `lo <= hi` to `lo < hi` in your `binary_search`. Which tests fail, and why exactly those?

The midterm window

Weeks 1–7 are examinable. For each week, can you:

- **1–2:** define an ADT; give Θ of a loop; prove a Big-O bound?
- **3:** write a recurrence and trace a call stack?
- **4:** explain amortised $O(1)$ append?
- **5–7:** draw a linked list, stack, queue or ring **after every operation?**

The **mock exam for weeks 1–7** is in the question bank.

Summary

Seven things to keep

1. Linear search needs no order and costs $O(n)$.
2. Binary search needs **sorted** data and costs $O(\log n)$: 20 comparisons for a million.
3. Its invariant: if the target is anywhere, it is in `values[lo..hi]`.
4. Get the boundaries right: `lo <= hi`, `mid ± 1` — test sizes 0, 1, 2.
5. Recursive: pass `lo` and `hi`; a slice turns $O(\log n)$ into $O(n)$.
6. `lower_bound/upper_bound` return **positions**: insertion points and counts of duplicates.
7. Jump $O(\sqrt{n})$, exponential $O(\log i)$, interpolation $O(\log \log n)$ average but $O(n)$ **worst**.

Week 9 — Basic sorting. Bubble, selection and insertion sort — and counting sort, which beats $O(n \log n)$ by never comparing at all.